The MWEB Grimoire

Rigo Salazar, Luke Szramowski

December 9, 2025

Change log

- December 9, 2025: Initial preprint

Contents

1 Introduction **2**
- 1.1 History of MWEB . . . 2
- 1.2 Motivation for MWEB . . . 2
- 1.3 Purpose of this Work . . . 3
- 1.4 Structure of this Work . . . 3

2 Preliminaries **3**
- 2.1 Notation . . . 3
- 2.2 Assumptions . . . 4
- 2.3 Hash Functions . . . 4
- 2.4 Pedersen Commitments . . . 5
- 2.5 Range Proofs . . . 5
- 2.6 Schnorr Scheme . . . 5
- 2.7 HD Wallet . . . 6
- 2.8 Merkle Trees . . . 6
- 2.9 Merkle Mountain Ranges . . . 6
- 2.10 Interactive Mimblewimble Protocol . . . 7
- 2.11 Non-Interactive Protocols . . . 7

3 General Overview **8**
- 3.1 What is an Extension Block? . . . 8
- 3.2 Extension Block Creation . . . 9
- 3.3 Moving into the Extension Block . . . 9
- 3.4 Transacting within the EB . . . 10
- 3.5 Moving out of the Extension Block . . . 11

4 Detailed Overview **12**
- 4.1 Terminology and Definitions . . . 12
- 4.2 Extension Block Structure and Specification . . . 13
- 4.3 Peg-In Transaction . . . 14
- 4.4 Internal EB Transactions . . . 15
- 4.5 Peg-Out Transaction . . . 19
- 4.6 MWEB In Action . . . 20

5 Security and Privacy Guarantees **25**
- 5.1 Inflation Resistance . . . 25
- 5.2 Theft Resistance . . . 25
- 5.3 Horizon Attack . . . 26
- 5.4 Kernel Malleability . . . 26

A Appendix **27**

1 Introduction

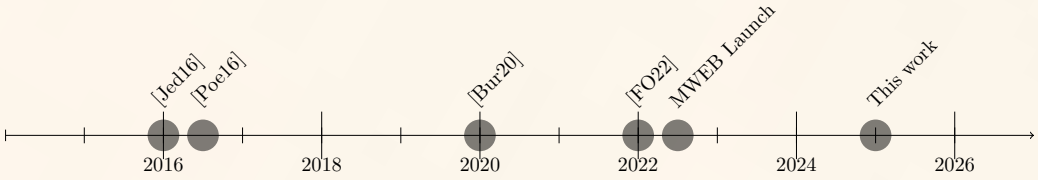
1.1 History of MWEB

In 1998, the term "**mimblewimble**" was first coined in the novel, *Harry Potter and the Chamber of Secrets*; it was a magic spell that prevented targets from speaking clearly. The phrase remained rooted in fiction until 2016, when an individual mysteriously released a document onto a Bitcoin research IRC channel. Said document outlined changes to a typical blockchain protocol, and was entitled Mimblewimble [Jed16].

The white paper, while interesting and novel, was incomplete. To remedy this, a mathematician named Andrew Poelstra filled in the gaps and corrected issues, publishing his own paper [Poe16] later that year while retaining the name. Around that time, a digital currency called Grin [Gri] began implementation of Mimblewimble, leveraging the privacy and scalability claims of the protocol. Epic Cash [Epi], a popular fork off of Grin, would also retain Mimblewimble as its primary protocol.

In the late 2010s, Mimblewimble did not allow for non-interactive transaction, which limited its popularity compared to other privacy coins. David Burkett, an implementer, attempted to rectify this in [Bur20]. With active collaboration with the privacy community via Grin keybase channels, this was further refined into [Bur21]. One particularly notable contribution was Minglejingle [Tev21], where stealth subaddresses and other applied lessons from Monero were integrated. Around early 2021, Litecoin, a digital currency with a market cap of $6.6 billion as of this writing, started integrating a variant of Mimblewimble into their protocol, with a full launch in 2022. This variant acted as a privacy layer on top of the established Litecoin blockchain, and was therefore dubbed *MimbleWimble Extension Blocks (MWEB)*

While there are more coins that use Mimblewimble, and other milestones in the development and testing of MWEB, we will cover the necessary information in later sections. Below is a timeline which records, in broad strokes, the mathematical focused papers that MWEB is built on.



1.2 Motivation for MWEB

For anyone familiar with privacy-based cryptocurrency, their necessity is constantly stated and restated. Obviously, in a world which increasingly encroaches on the personal lives of individuals, it is important to have tools to push back. Coins like Monero and Z-cash are held in high regard because of their intense devotion to decentralization and privacy, but the often-sidelined topic of use case is also important.

The reality is that the majority of the first-world is already accepting of what we coin, "*debit card security*", where an individual knows that their immediate peers cannot view their financial history, but are aware that entities like governments and banks do have that ability. Of course the coins mentioned prior fight tooth and nail to remove that vulnerability, but that comes at a cost. Many common exchanges do not support Monero, which is private by default. Even Firo was recently deslisted from Binance in early 2025, despite attempting to accommodate Binance's desire for transaction risk analysis [Yap25]. Potential users are forced to jump through more hoops to use privacy coins, and that unfortunately deters many people from using them.

Of course, if there is a need, then a market will exist to satisfy it. Because MWEB is built on top of the user-friendly Litecoin, which plays well with major exchanges, it is factually more accessible to the average individual. Additionally, the average individual is typically not concerned with a government-level adversary, they just don't want their co-workers to see their account balance when paying them back after a friendly lunch. This juxtaposition belies the fact that privacy has gradations. A privacy elitist would scoff at sacrificing their personal information for accessibility, but they simply don't make up the entire population.

For most, as long as an e-coin is as private as a debit card, then that is sufficient. We believe that MWEB not only fits that niche, but does so impressively.

1.3 Purpose of this Work

While there have been several instances of formalizing Mimblewimble and the subsequent advancements like [Yu20] and [FO22], it has come to the attention of Cypher Stack that there is a lack of documentation specifically covering MWEB as it is deployed in Litecoin. Indeed, [FO22] makes it a point to mention that MWEB is a variant, but only vaguely mentions some broad differences. It is the desire of Cypher Stack that, upon reading this paper, any mathematician (or individual with a technical background) would be able to get "up to speed" with MWEB. This includes its goals, security guarantees, mathematical proofs, and construction process.

1.4 Structure of this Work

In Section 2, we go over the building blocks of MWEB. This lengthy section includes the relevant mathematical notation and cryptographic primitives. Section 3 has a topical description of the hopping between the extension block and main block, while Section 4 dives into a more mathematical description. Section **??** discussed what a transaction in the extension block actually looks like, and Section 5 describes the guarantees that MWEB offers.

2 Preliminaries

2.1 Notation

Definition 2.1: Notation

Symbol	Meaning
$\mathbb{N}$	The natural numbers.
$\mathbb{N}_0$	The natural numbers, including 0.
$\mathbb{Z}$	The integers.
$\mathbb{Z}_p$	The integers modulo p.
$[n]$	The set of numbers from 1 to n, where n is a natural number.
$x \xleftarrow{\$} S$	x is sampled uniformly at random from S
$\mathbb{G}$	A prime group[a] of order p.
G, H	Generators of the group $\mathbb{G}$.
$(A \mid\mid B)$	Concatenation of the strings/vectors A and B.
$\mathcal{H}(\cdot)$	A general cryptographic hash function.
$X[:N]$	A byte string X truncated at N bytes
$\mathbb{F}_p$	A field with p elements.

[a] In MWEB, the group in question is the elliptic curve *Secp256k1*

In general, capital letters will denote elements of $\mathbb{G}$, whereas lowercase letters will denote elements of $\mathbb{F}_p$. The generators G and H will also have the additional assumption that there are no known discrete logarithm relations between the two of them, which is necessary for the security guarantees of the Pedersen commitment. In the case of summations present, a slight abuse of notation is allowed for brevity: if the summation does not possess indexes, it should be assumed that the sum spans over all mentioned elements of the type described inside the sum.

2.2 Assumptions

We highlight several assumptions on which we will base our security, namely the generalized discrete log problem, the Diffie-Hellman problem, and the problem of finding hash collisions. We assume that these are intractable for a computationally bound adversary.

Assumption 2.1: The Discrete Logarithm Problem

Given $G \in \mathbb{G}$ and $P = x \cdot G$ for some $x \in \mathbb{F}_p$, then it is computationally intractable to recover x.

Assumption 2.2: The Decisional Diffie-Hellman Problem

Given $G \in \mathbb{G}$, $x \cdot G$, and $y \cdot G$, it is computationally intractable to distinguish $x \cdot y \cdot G$ from $z \cdot G$, where $x, y, z \xleftarrow{\$} \mathbb{F}_p$

The security of Assumption 2.2 relies on Assumption 2.1, in that if one can efficiently solve the discrete logarithm problem, then they can solve Assumption 2.2 below. In other words, solving the latter is harder than the former. That isn't to say that algorithms don't exist for that purpose, although they are limited by time. In a pre-quantum setting, general attacks like Pollard's rho algorithm [Pol10] can be referenced for expected time.

2.3 Hash Functions

For security purposes, we also have requirements for hash functions. Below are some definitions that we require for hash functions to be useful in a cryptographic context.

Definition 2.2: (Strong) Collision Resistance

Given $m_1, m_2 \in \mathcal{M}$ (where $\mathcal{M}$ is some message space) such that $\mathcal{H}(m_1) = \mathcal{H}(m_2)$, the probability that $m_1 \neq m_2$ is bounded by some negligible function for all probabilistic, polynomial-time adversaries.

Strong collision resistance is a cryptographically desirable property for a hash function to possess and, more often than not, a necessary property to prove security results related to schemes requiring hash functions. We also require preimage resistance, which is distinct from and not implied by collision resistance, which states that it's difficult to calculate preimages.

Definition 2.3: First Preimage Resistance

We assume that finding a hash's preimage is hard. That is, to say, that given $\mathcal{H}(m)$ for some $m \in \mathcal{M}$, then any probabilistic, polynomial-time adversary $\mathcal{A}$ has a negligible advantage in identifying m.

There are a variety of different hash functions to choose from, like Blake2b [ANWOW13] or Keccak [BDH$^+$], but MWEB uses the BLAKE3 [jo20]. In Section 4, all usage of $\mathcal{H}(\cdot)$ should exclusively be read as the BLAKE3 hash function.

Extra care should be taken when analyzing cryptographic schemes involving hash functions. Namely, the domain (set of inputs) and the range (set of outputs), should be considered when looking at its application. There are some hash functions which produce points which lie on an elliptic curve, whereas others might produce elements from the *underlying field* of an elliptic curve. In our case, the output for BLAKE3 is a 32-byte scalar modulo the curve order.

2.4 Pedersen Commitments

MWEB makes proficient use of Pedersen commitments, which are the most common commitment scheme for zero-knowledge protocols. To commit to $v \in \mathbb{G}$, we take $r \xleftarrow{\$} \mathbb{G}$ and set $C = \mathsf{Com}(v, r) := v \cdot H + r \cdot G$. In MWEB, v represents the value of Litecoin used in a transaction, while r is a private key used as a blinding factor. Commitment schemes rely on the following properties:

Definition 2.4: The Hiding Property

A commitment scheme is *computationally hiding* if it is infeasible to determine the secret v from $\mathsf{Com}(v,\, r)$. More precisely, if $v_1, v_2 \in \mathbb{G}$, and there is a commitment $\mathsf{Com}(v_X,\, r)$ where X is 0 or 1, then there is only a negligible chance that a probabilistic, polynomial-time adversary $\mathcal{A}$ can correctly identify X better than a fair coin flip.

If any adversary $\mathcal{A}$ is unbounded in their capabilities, but they still cannot correctly identify X from the commitment, then we say the scheme *perfectly* hiding.

Definition 2.5: The Binding Property

We say that a commitment scheme is *computationally binding* if the probability of finding $(v_1,\, r_1) \neq (v_2,\, r_2)$ such that $\mathsf{Com}(v_1,\, r_1) = \mathsf{Com}(v_2,\, r_2)$ is very low. If this probability is exactly 0, then we say the scheme is *perfectly* binding.

Pedersen commitments are perfectly hiding and computationally binding, whereas other schemes, like ElGamal, are the opposite [ElG85]. No commitment scheme can be both perfectly hiding *and* perfectly binding; there will always be a trade-off. Many other cryptocurrencies like Firo [MGGR] and Monero [Noe15] use Pedersen commitments, so MWEB is in good company. We note is that Pedersen commitments enjoy binding at the behest of Assumption 2.1. As an aside, MWEB also has a switch commitment in place to transition from Pedersen to ElGamal for a post-quantum world. This switch commitment can be activated via miner signaling.

Of particular importance in the MWEB scheme is the fact that Pedersen Commitments are homomorphic with respect to the inputs, i.e. $\mathsf{Com}(v_1, r_1) + \mathsf{Com}(v_2, r_2) = \mathsf{Com}(v_1 + v_2, r_1 + r_2)$. This will be extremely useful because Mimblewimble relies on adding and subtracting commitments in order to obscure transaction amounts. As a last note, in this document we write Pedersen commitments additively.

2.5 Range Proofs

If the amount of money is obfuscated (by Pedersen commitments in the case of MWEB), then there needs to be some assurance that a user isn't fabricating additional money that never existed. This could be done by using a negative v, which a computer would read as an enormous number because of the modular nature of the elliptic curve group properties. During most of the initial development of MWEB, Bulletproofs [BBB$^+$17] were the tried-and-true option and are thus the range proof of choice. Although newer versions like [CHJ$^+$20] and [EKRN22] have been published since, MWEB developers have opted not to switch because the messiness of a hard fork is arguably not worth the benefits as of this writing [Bur25].

2.6 Schnorr Scheme

Patented in 1990, Claus Schnorr invented a digital signing algorithm he named after himself. Basing its difficulty on Assumption 2.1, the Scnorr signature protocol is an efficient algorithm that pops up often enough in MWEB that it warrants a mention. See below:

Diagram 2.1: Schnorr Signature Scheme

Reading from top to bottom should be read as the sequence of events with the middle arrow(s) indicating communication between parties.

Public: $p \in \mathbb{N}$, $G \in \mathbb{G}$, $X = x \cdot G$ **Private:** $x \in \mathbb{F}_p$		
Prover		**Verifier**
Pick message M		
Select $r \xleftarrow{\$} \mathbb{F}_p \setminus \{0\}$		
Compute $R = r \cdot G$		
Set $e = \mathcal{H}(R\|\|M)$		
Let $s = r - xe$		
Let $\sigma = (s, e)$	$\xrightarrow{\sigma}$	
		Let $R' = (s \cdot G)(e \cdot X)$
		Set $e' = \mathcal{H}(R'\|\|M)$
		Check $e \stackrel{?}{=} e'$

2.7 HD Wallet

Shorthand for *hierarchical deterministic wallet*, an HD wallet is an e-coin wallet that is capable of generating private and public key pairs from a *seed*. Computers have a much easier time remembering absurdly long bit strings, but humans aren't built the same. For convenience, a seed takes the form of a list of words (between 12 and 24 for Litecoin) which becomes the *master key*. The master key deterministically generates addresses which can deterministically generate sub-addresses, and so on. The generator and what is generated establish a hierarchy for an e-coin wallet. Diagrams of HD wallets are often likened to a tree.

2.8 Merkle Trees

As if it weren't confusing enough, there is another type of tree to discuss. Merkle trees are data structures that uses hash functions to verify the authenticity of large swaths of data. A direct example in a cryptographic context is a set amount of transactions. Each transaction would be hashed and would then be referred to as *leaf nodes*. These leaves would be paired with each other, and then hashed again to form a *parent node*[1]. Parents would then be paired and hashed once more to continue the process. The components of the pairs are also referred to as *child nodes*. This procedure continues until there are only two remaining parent nodes that are hashed together to create a single *root*. Observers can verify data validity by comparing hashes at different levels of the tree. This works because of Definition 2.2, so even slightly altering one transaction would create rippling differences throughout the branches and ultimately the root.

2.9 Merkle Mountain Ranges

While Merkle trees are efficient, they can be limited when new data must be added to the dataset. For a blockchain that is constantly being updated, this is a grave setback. A Merkle mountain range ([MMR]) is a data structure similar to the Merkle tree, but nodes are constantly being added left-to-right. Once two children exist (at any height), the next node is the parent, and the process continues. This can be hard to visualize, so please refer to Figure 1 where each integer represents the nodes in order of creation:

[1] An odd number of transactions would be handled by choosing one leaf and hashing it with itself

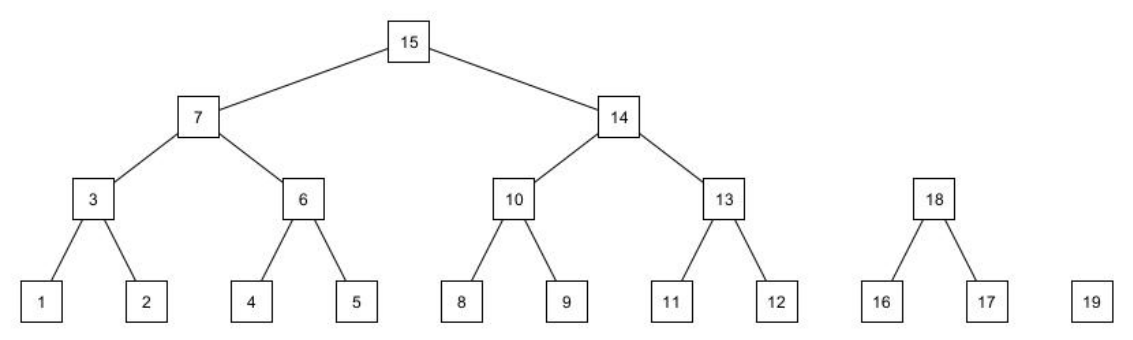


Figure 1: MMR Diagram

Since new nodes are constantly being created, there is no root for the MMR, per se. So when attempting to verify the data authenticity like in a normal Merkle tree, a process called *batching the peaks* takes place. In the above graphic, the nodes 15, 18, and 19 are the *peaks* of the MMR, and are hashed together to create the root equivalent. Verifiers can then authenticate data by checking the hash. Thus, MMR's maintains the verifiability while adding a flexibility that typical Merkle trees cannot match.

2.10 Interactive Mimblewimble Protocol

As mentioned earlier, one of the greatest drawbacks of the original Mimblewimble protocol was that interactivity was not optional. This has obvious limitations in an e-coin setting because both the sender and receiver would need to communicate off-chain to perform a Diffie-Hellman exchange and the money would have to manually change hands. There has been an abundance of research papers dedicated to eliminate the back-and-forth of Mimblewimble while retaining the scalability and privacy guarantees. [FO22] was able to correct several issues in prior works and still retain the *cut-through* property, for example. MWEB unfortunately forgoes cut-through because the it is at odds with the stealth excess checks. Even though MWEB does not require interactivity to function, we include a diagram in Appendix A showcasing the protocol because the "spirit" is still very present. Since this is the general Mimblewimble Protocol, we wish to clarify that the hash functions present are not necessarily BLAKE3.

2.11 Non-Interactive Protocols

The obvious counterpart to an interactive protocol, a non-interactive protocol is ideal for the blockchain currency use case. Interactivity implies a constant back and forth where the prover and verifier send variables back and forth. Verification can be assured because the blueprint for the protocol dictates that the parties use the shared variables in such a way that there is no question as to the legitimacy of the protocol. A non-interactive protocol, on the other hand, has a single communication: the prover posts a full transcript and the verifier sees said transcript and checks it. Because there isn't a general sharing of variables and commitments (and their respective dependencies), there needs to be another way to assure a verifier that no "funny business" is occurring within the transcript. The most common method is to use hash functions in such a way that the verifier can easily recompute the exact values that the prover used in their transcript. To avoid attacks like the Frozen Heart vulnerability [Mil], it is imperative to make sure that the protocol includes **all** relevant information inside of the hash, lest a malicious prover back-calculates specific values to give themselves a trap-door into their transcript at a later time. As stated above, MWEB removes the interactivity; once a user wishes to spend the coin they generate the transcript and leave it to be verified with no further action on their end.

3 General Overview

3.1 What is an Extension Block?

An extension block can be thought of as an additional block which functions alongside the main blockchain, but follows a different set of rules. It is an entirely optional structure; users do not have to interact with it if they do not see the benefit in adhering to the different set of rules.

There can be any number of extension blocks that can be 'attached' to a blockchain, each differing by the rules that the creator wishes to enforce. They will all possess some similar properties, namely:

Property 3.1: Extension Block Properties

An *Extension Block* must provide the following:

1. There must be a way for users to send coins in to the extension block, which is referred to as **pegging-in**.
2. There must be a way for users to send coins out of the extension block, which is referred to as **pegging-out**.
3. There needs to be a one-to-one correspondence between peg-in transactions on the main chain, and coinbase transactions on the extension block.
4. The extension block must be able to be committed to with a single hash.

The above properties are needed to make an extension block usable. The first two properties are obvious: users must have a way to get in and get out of the block. Once there, all that remains is to simply follow the rules that are predefined within the block, which will be discussed later.

The third property is less obvious: a coinbase transaction is the method by which new coins are minted. So, if a user pegs-in to an Extension Block, it is not as if their coins on the main chain are transmuted into coins in the Extension Block. Rather, the coins that are pegged-in are made unavailable to them on the main chain and an equal amount of coins are dispensed on the Extension Block, which can be used to transact within the block.

The fourth property allows for an easy way of committing to the extension block, and such commitments will be used in the detailed protocol later on.

There is quite a bit of utility that can be gained for using an Extension Block, namely:

- It allows two different types of transactions, the ordinary type on the main chain and the transactions which take place within the block.
- It does not hinder the experience of people who wish to remain only on the main chain, since a user must purposefully *opt-in* to use it.
- The transactions on the main chain and on the extension block are kept separate.
- It can be used to increase the block size of the blockchain, which is of great utility to currencies that have a fixed block size.

The natural question to be asked is: how does MWEB achieve this? The general method can be described by asking and answering the following questions:

1. How are the Extension blocks created and maintained?
2. How does one go about placing funds in the Extension Block?

3. What do transactions within the Extension Block look like?
4. How does one retrieve funds when one wants to leave the Extension Block?

The following sections supply the answers to each of these questions.

3.2 Extension Block Creation

The first thing that needs to be understood is that the extension block is not part of some parallel running blockchain. It is simply additional data which is appended to the already existing blockchain, in such a way that the additional data has its own structure and rules which it follows.

In the case of MWEB, the additional structure and rules of the extension block are those that allow use of the Mimblewimble protocol. So, when referring to the Mimblewimble Extension Block, we are really talking about this extra bit of data contained within the main block chain. The is advantageous in that worrying about two concurrently existing chains and their potential interactivity issues can be sidestepped.

The extension block is going to have all of the components needed for Mimblewimble transactions to take place:

1. The set of all the unspent outputs from the previous 'block'. These are effectively the MWEB 'coins'.
2. The set of transactions that occurred within the block.
3. The special address which contains all peg-in funds.
4. Various technical pieces used to allow peg-ins, peg-outs and transactions (which will be explored in the Detailed Overview section).

3.3 Moving into the Extension Block

While the preceding section is a bit esoteric and not entirely concrete, the process of moving into the extension block, referred to as *pegging-in*, yields the needed context to paint a more complete picture. Suppose a user has x amount of coins that they wish to 'move' into the extension block. What exactly happens?

Recall that in Mimblewimble, all the inputs and outputs are Pedersen commitments of the form:

$$C = v \cdot H + \rho \cdot G$$

where v is the values and ρ is some randomness known only to the owner. Everyone can see C, but only the owner can reveal information about it, since it is obfuscated via Assumption 2.1. So, when the user pegs-in, they send your actual coins to a special wallet. Since the coins are no longer in the user's wallet, spending the coins is prohibited, in effect the coins are locked away. At the same time, the peg-in process will produce a Pedersen commitment to the amount x and provide the user with the randomness r:

$$C_{MW} = x \cdot H + r \cdot G$$

Now everyone can see that C_{MW} is a 'coin' within the extension block, but only the specific user has access to the randomness, which effectively gives them ownership of the coin We can note that viewers can infer x since a new MWEB coin was assigned, but they can't determine the r value. Imagine the peg-in process as a token dispenser at an arcade. The coin is only useful inside the extension block (like the token is in the arcade) and once possessed, Mimblewimble transactions are possible, with all of the privacy that it provides.

One potential misconception that should be addressed is that these created funds do not magically possess value via some POW scheme like ordinary cryptocurrency does. Rather, they *inherit* the value of the coins that are locked up, since they are no longer spendable and cannot be withdrawn *unless* a peg-out transaction is performed. A user must prove that they can withdraw (or unlock) the funds from the extension block by

having a Pedersen commitment which exists in the extension block. This provides security against fraudulent withdraws.

Now, the natural question arises, "Well, what is stopping a malicious attacker from making some Pedersen commitment $X = 100 \cdot H + 50 \cdot G$, taking it to the miners and asking them to peg-out for 100 Litecoin?" Notice, the first thing that the miners are going to do is check and see if X is even in the pool of recorded unspent outputs. Since X was pulled one out of thin air, the probability that you made X really exists in that pool is negligibly small. The only people which can actually withdraw funds are people who have total knowledge of the coin, that is to say, the amount and the randomness.

3.4 Transacting within the EB

Once in possession of your extension block funds, users are ready to perform Mimblewimble transactions with other people also inside the extension block. How exactly does this process go about? It is easiest to take a look at a concrete example. Let's consider two MWEB users, Albus and Barty, who wish to perform a transaction. Albus has 20 MWEB coins and wishes to pay Barty 15 of them.

Step 1

Since Albus has pegged-in 20 Litecoin, he is in possession of a Pedersen commitment of the form:

$$C_A = 20 \cdot H + r \cdot G$$

where C_A is in the unspent transaction set of the extension block. If he wants to make a coin with a value of 15 for Barty, he simply makes a transaction from himself *to* himself. By knowing r, this is very easy to do. He can produce coins C_{spend} and C_{change} where:

$$C_{spend} = 15 \cdot H + r_1 \cdot G$$

$$C_{change} = 5 \cdot H + r_2 \cdot G$$

We ultimately want the sum of the inputs (C_A) minus the sum of the outputs (C_{spend} and C_{change}) to be equal to zero. This is important because it guarantees that Albus didn't magically conjure money out of thin air.

$$\underbrace{20 \cdot H + r \cdot G}_{C_A} - (\underbrace{15 \cdot H + r_1 \cdot G}_{C_{spend}} + \underbrace{5 \cdot H + r_2 \cdot G}_{C_{change}}) = 0$$

Canceling out the like terms, we get that:

$$r \cdot G - (r_1 \cdot G + r_2 \cdot G) = 0 \Rightarrow (r - r_1 - r_2) \cdot G = 0$$

To make everything work out nicely, all Albus needs to do is ensure:

$$r = r_1 + r_2$$

Then everything will cancel out nicely and the transaction will be balanced and then recorded in the unspend outputs set. Albus can do this behind the scenes, without any input from Barty and ultimately, Albus will be in possession of two coins C_{spend} and C_{change}, both of which are recognized as valid coins in the extension block.

Step 2

For a classical Mimblewimble transaction, Albus needs to transfer ownership of the C_{spend} coin to Barty. Obviously, Albus could just tell Barty the randomness (r_1) associated with the coin and since the amount is known by both, Barty would technically have ownership of the coin. The issue lies in the fact that Albus also knowing the r_1 means he still has spend access, outflanking Barty's control.
To circumvent this issue, Barty is going to choose an additional randomness (r_3), that Albus doesn't know. So, the transaction is going to have an input of C_{spend} and an output of C_B. They will look like:

$$C_{spend} = 15 \cdot H + r_1 \cdot G$$

$$C_B = 15 \cdot H + r_3 \cdot G$$

Of course, unless the randomness of both are the same, the transaction will never balance to zero. If we take the difference of the input and output, though, we are left with:

$$C_{spend} - C_B = r_1 \cdot G - r_3 \cdot G = \underbrace{(r_1 - r_3)}_{\textbf{excess}} \cdot G$$

This **excess** term would be problematic, but Albus and Barty can then make a signature for this term, which they include with the transaction. Barty will now have his coin, C_B and a 'certificate' stating that the excess amount is equal to the above quantity. Subsequently, anyone can verify that the transaction sums to the excess and after removing the excess amount, sums to zero. This keeps the balance and now Albus doesn't know the randomness of Barty's new coin, thus completing the transfer of ownership.

This is the end of the transaction using the classic Mimblewimble technique. Actual transactions in the extension block are a little different, but follow the same general premise when in the block. As we will see in the more detailed version, transactions will include more data to ensure that the pegging-out process is smooth and that funds are dispensed properly.

3.5 Moving out of the Extension Block

After some transactions, a user might decide to withdraw some of the money present in the extension block back out into the main chain. The procedure is simple:

1. Suppose that you have a coin worth x in MWEB and you want to withdraw y worth of value.
2. Similar to what Albus did in step one, you will make two transactions with yourself: one coin worth y (say, C_y), and one coin worth $x - y$ (say, C_{x-y}).
3. You perform the peg-out process with C_y, which signals to the miner to move y locked coins from the special address to the Litecoin address specified in the peg-out process.
4. Once the block is mined, the funds are dispensed and the C_y no longer becomes available for use in the extension block.

One might refer to the last step as the 'burning' of funds, but really, C_y is just an unspent output which is included in the set of all unspent outputs. So, all the miners have to do to ensure it can no longer be accessed is to just remove it from the set of unspent outputs.

4 Detailed Overview

This section will follow the exact same format as the previous one, but with a slight caveat: all the rigorous mathematical details that were omitted there will be included here. We will take extra care to detail each object and piece of notation, as well as how it will all fit together. We will begin with a short overview of notation and diction that the reader may not be familiar with.

4.1 Terminology and Definitions

The first bit of terminology we need to discuss is one that is integral to the original Mimblewimble protocol. A *transaction kernel*, sometimes just referred to as a *kernel*, is the method by which transactions within Mimblewimble can remain balanced while also allowing safe transactions in the form of Pedersen commitments.

Definition 4.1: Transaction Kernel

A *transaction kernel* for a Mimblewimble transaction, is a signature, commitment and fee triple, which guarantees that two transacting parties can jointly produce the excess values produced by the receiver's new blinding factor, as well as the ensuring that the balanced sum of the transaction are zero.
We can express a transaction kernel as (σ, C, f), where σ is the signature, C is the commitment to the excess value and f is the fee amount.

This commitment is going to act as a *public key* for the transaction. Anyone who wishes can take the difference of the input values and output values and compare them to the public key, which for a valid transaction, will always be equal. The transaction kernel can be represented in the following format:

[**Type**][**Fee**][**Commitment**][**Signature**]

The size of a transaction kernel is 102 bytes: 1 for the type, 4 for the fee, 33 for the commitment and 64 for the signature. Currently there are 6 different kernel types, each specified as follows:

Type	Bit Indicator
Fee	0x01
Peg-in	0x02
Peg-out	0x04
Height Lock	0x08
Stealth Excess	0x10
Extra Data	0x20

These indicators signal within the code the type of information stored in the kernel, and also determine how the signature is formed for a particular type.

While the following definition is not necessarily an integral part of what make Mimblewimble work, it is still used in the *stealth address* portion.

Definition 4.2: Derivation Paths

A *derivation path* references the location of keys within an HD Wallet. The notation of the derivation path is:

$$m/a/b/c$$

where:

1. m denotes the deterministically generated master key.
2. Each '/' denotes a new level in the tree.
3. a, b, c denote the index number of the child from the parent.
4. When an h or a ' is attached to the number, this indicates a *hardened child*. This simply means that the index from which to start is the scalar 2147483648.

This may also be referred to as a *keychain path*.

4.2 Extension Block Structure and Specification

In the general overview, we simply described the extension block as an extra bit of data appended to the end of the main blockchain, which contains the pieces we need to use Mimblewimble to transact. In this subsection, let us be precise when details all of the components.

Component 4.1: Extension Address (ExtAddr)

The *extension address*, more formally known as the **Hogwarts Address** (*HogAddr*), is the address in which all pegged-in funds are stored. Each extension block has an extension address which is different from the previous one and each address will contain exactly the same number of coins as the number that exist within that extension block.

The extension address is a sort of storage for all the coins which are pegged-in and the wallet from which transactions are pegged-out from. When a user pegs-in, they send their funds to a special address type, which miners then use to send those funds to the HogAddr of the extension block. The special address type is defined as follows:

Component 4.2: Bech32 Addresses

A *bech32 address* is a special addressing format which is used for new segregated witness (segwit) locking scripts. It can be identified with the starting prefix *bc*1 and possesses superior qualities over other segwit address formats, including:

1. Identification of whether or not the address was entered correctly.
2. Built in error correction capabilities for incorrectly entered addresses.
3. Superior encoding and decoding.
4. Increased human readability.

The address type is chosen because it supports segwit scripts, which have benefits regarding transactions which have components still stored in the mempool (unmined transactions). The address generated will be

an *anyone can spend* address, which is what allows miners to collect these addresses and send them to the *HogAddr* of an extension block.

Component 4.3: Extension Block Commitment

An *extension block commitment*, specified in the implementation as *eb_commitment* is a commitment to the following three pieces of data:

- The sum of all the kernel offsets within the block.
- The roots of the merkle mountain range of MW transaction kernels in the extension block.
- The roots of the merkle mountain range of the unspent transactions in the extension block.

We can think of this as a way of committing to a specific extension block at a certain point in time. Lastly, within the extension block is contained the pieces of data necessary to make and verify transactions. This will include:

1. The set of outputs, which are the 'coins' inside the extension block. An ordinary Mimblewimble output is just a Pedersen commitment with an accompanying rangeproof, however, more information may be appended in the case of stealth addresses.
2. The transaction kernels, as defined above.
3. A Merkle mountain range for the output set.
4. A Merkle mountain range for the rangeproofs corresponding to the outputs.
5. A Merkle mountain range for the transaction kernels.

4.3 Peg-In Transaction

We can think of the process of pegging-in as one that achieve the following goals:

- The funds to be pegged-in are prepared to be sent to the *HogAddr*.
- A new output is added to the set of outputs within the extension block.
- The data within the output is appended to the appropriate Merkle mountain ranges.

In order to achieve this, the peg-in transaction starts with sending the desired peg-in amount to a special bech32 address which is indicated by the *ltcweb1* prefix at the beginning. All addresses tagged with this prefix are collected by the miners, who will then spend the pegged-in amount within the address, by making a transaction with the *HogAddr*. This processes is referred to as the *Integrating Transaction*, or the *Hogwarts Express* (HogEx).

Part of this integrating transaction is the creation of the kernel commitment to the peg-in kernel (Definition 4.1), which is effectively a commitment to the peg-in amount contained in the kernel. This commitment will be the spendable currency within the extension block and is the 'coin' that is released to the user.
Lastly, the miners will append the necessary data to the Merkle mountain ranges within the extension block, thus completing the integrating transaction and pegging-in the user.

4.4 Internal EB Transactions

The feature that MWEB boasts which classical implementations of Mimblewimble lack is the capability to make one-sided transactions inside the Extension Block. This introduces non-interactivity which MW does not natively possess. Additionally, implementing MW in the extension block is simply one feature that the extension block possesses. It is possible to implement additional features that users who transact within the extension block can benefit from.

Stealth Addresses

MWEB introduces the ability to generate stealth addresses to use in the extension block. These stealth addresses are generated from the wallet seed in a deterministic manner.

Definition 4.3: Stealth Addresses

A *stealth address* is defined as the pair (A_i, B_i), where $A_i = a_i \cdot G$ and $B_i = b_i \cdot G$. A_i is referred to as the *scan key* and B_i is referred to as the *view key*.

Stealth addresses are generated deterministically from the wallet seed. They are generated in the following way:

Algorithm 4.1: Stealth Address Generation

Given a wallet seed S:

1. Using HD keychain path $m/1/0/100'$, generate the *master scan key-pair*:

$$(\mathbf{a}, \mathbf{A})$$

2. Using HD keychain path $m/1/0/101'$, generate the *master spend key-pair*:

$$(\mathbf{b}, \mathbf{B})$$

3. Choose the lowest unused address index **i**.

4. Compute the one-time spend key-pair: (b_i, B_i) as:

$$b_i = \mathbf{b} + \mathcal{H}(\mathbf{A} \mid\mid i \mid\mid \mathbf{a})[:32]^a$$

$$B_i = b_i \cdot G$$

5. Compute the one-time scan key-pair as:

$$a_i = \mathbf{a} \cdot b_i$$

$$A_i = a_i \cdot G$$

This will produce the stealth (sub)address (A_i, B_i).

[a] BLAKE3 hash outputs are 32-byte by default, but we follow the notation of [Bur21] since the protocol truncates hashes at differing sizes depending on context

It should be noted that the number of stealth addresses that one can generate in this way is incredibly large, so there is no concern that one will ever run out of viable stealth addresses. Additionally, in the above algorithm, Step 3 prevents stealth address reuse, which is of utmost importance in preserving privacy when using them.

Now, let us take a look at the MWEB transactions, which will consist of 4 major pieces: the inputs, the outputs, the kernels and the kernel offsets.

Definition 4.4: MWEB Transaction Inputs

An *MWEB transaction input* is a tuple:

$$[K_i, C, K_o, \sigma]$$

where:

1. K_i is an ephemeral key of the form:
$$K_i = k_i \cdot G$$
where $k_i \xleftarrow{\$} \mathbb{F}_p$.
2. C is a previously unspent output which the transactor owns.
3. K_o is the output public key corresponding to C.
4. σ is a Schnorr signature of $\mathcal{H}(C)[:32]$, with input key:
$$K_{input} = K_i + \mathcal{H}(K_i \,||\, K_o)[:32] \cdot K_o$$

The transaction input is simply the money that you wish to send to a receiver. The above input is set up in such a way that it can be publicly posted, as part of the non-interactive setup.

Definition 4.5: MWEB Transaction Outputs

An *MWEB Transaction Output* is a tuple:

$$[C, \pi, v', n', K_s, K_o, K_e, \rho]$$

which is associated with an output address (A_i, B_i), where:

1. A coin C, which has the form of the Pedersen Commitment:

   $$C = v \cdot H + q \cdot G$$

   where q is the opening which is shared between the sender and receiver and v is the value of the coin.

2. π is a range-proof which ensures C commits to some $v \in [0, v_{max}]$ and also commits to the message:

   $$(C \;||\; v' \;||\; n' \;||\; K_s \;||\; K_o \;||\; K_e)$$

3. The masked value, v', which is the ciphertext of value v, encrypted by the shared secret t.

4. The masked nonce, n', which is the ciphertext of a one-time nonce n, encrypted by the shared secret t.

5. K_s is an ephemeral key chosen by the sender:

   $$K_s = k_s \cdot G$$

6. K_o is the output public key:

   $$K_o = k_o \cdot G$$

   where k_o is known only to the receiver.

7. K_e is the key exchange public key:

   $$K_e = s \cdot B_i = (sb_i) \cdot G$$

   where s is known to both the sender and receiver, but b_i is known only to the receiver.

8. ρ is a signature of the above message, with verification key K_s.

We should note here, for the sake of clarity, that the key exchange public key is derived from a Diffe-Hellman Key exchange which takes place between the users transacting. Additionally, the signature included is a Schnorr signature.

Definition 4.6: MWEB Kernels

An *MWEB Kernel* is a tuple:

$$[a, f, E, E', \psi]$$

where:

1. a is the pegged-in amount.
2. f is the fee for the transaction.
3. E is the excess:
$$E = \left(\left[\sum C_{out}\right] + f \cdot H\right) - \left(\left[\sum C_{in}\right] + x \cdot G\right)$$
where x is the transaction's offset and C_{out}, C_{in} are the output and input commitments, respectively.
4. E' is the stealth excess:
$$E' = \sum K_s + \sum K_i - \sum K_o - x' \cdot G$$
where x' is the transaction's stealth offset.
5. ψ is a signature for the message $(a \mid\mid f)$, where:
   - For transactions **with** stealth excesses, the signature must be valid under the verification key:
   $$\mathcal{H}(E \mid\mid E')[:32] \cdot E + E'$$
   - For transactions **without** stealth excesses, it must be valid under the verification key E.

We can think of MWEB kernels as a specific instantiation of a transaction kernel. They are formatted in a way to allow easy tracking of the excess amounts.

Definition 4.7: Transaction Offsets

A *transaction offset* is the pair of scalars (x, x'), where:

$$x \cdot G = \left(\sum C_{in} - \sum C_{out}\right) + (f - a) \cdot H - \sum E$$

$$x' \cdot G = \sum K_s + \sum K_i - \sum K_o - \sum E'$$

Algorithm 4.2: Input Creation

For each input:

1. Choose a sender key, $k_s \xleftarrow{\$} \mathbb{Z}_p$.
2. Generate a Schnorr signature σ, with message $m = \mathcal{H}(C_{in})[:32]$, under the key:
$$K_{input} = K_i + \mathcal{H}(K_i \mid\mid K_o)[:32] \cdot K_o$$

We now detail generating an output for a value v, which is sent to a receiver's stealth address (A_i, B_i).

Algorithm 4.3: Output Creation

1. Choose a sender key, $k_s \xleftarrow{\$} \mathbb{Z}_p$, and generate $K_s = k_s \cdot G$. This produces the sender's key-pair (k_s, K_s).
2. Compute the following:
   - The nonce:
   $$n = \mathcal{H}('N' \mid\mid k_s)[:16]$$
   - The sending key:
   $$s = \mathcal{H}('S' \mid\mid A_i \mid\mid B_i \mid\mid v \mid\mid n)[:32]$$
   - The shared secret:
   $$t = \mathcal{H}('D' \mid\mid s \cdot A_i)[:32]$$
3. Compute the receiver's one time public key:
   $$K_o = B_i \cdot \mathcal{H}('O' \mid\mid t)[:32]$$
4. Compute the key exchange public key:
   $$K_e = s \cdot B_i$$
5. Encrypt the following:
   $$v' = v \oplus \mathcal{H}('Y' \mid\mid t)[:8]$$
   $$n' = n \oplus \mathcal{H}('X' \mid\mid t)[:16]$$
6. Generate the signature ρ for message $(v' \mid\mid n' \mid\mid K_s \mid\mid K_o \mid\mid K_e)$ with sender's key k_s.
7. Compute the commitment:
   $$C = v \cdot H + q \cdot G$$
   where $q = \mathsf{SWITCH}(v, \mathcal{H}('B' \mid\mid t)[:32])$.
8. Generate the rangeproof π, proving $v \in [0, v_{max})$ and committing to the above message.

This will produce the transaction output:

$$[C, \pi, v', n', K_s, K_o, K_e, \rho]$$

4.5 Peg-Out Transaction

Lastly, moving funds out of the extension block is relatively straightforward. A user will make a peg-out transaction, which will contain the peg-out identification tag. The output which is being pegged-out will be identified within the set of outputs and the data associated with it will be identified on the various Merkle mountain ranges.

A transaction from the *HogAddr* to the users Litecoin wallet will be made, which will release the pegged-out amount in Litecoin to the user. Once this is done, all information regarding the pegged-out amount will be pruned from the identified sources prior, completing the peg-out process.

4.6 MWEB In Action

In order to fully appreciate how MWEB functions, it is necessary to look at an example (albeit a small one) of how the transactions within the extension blocks occur. An example is best explained in snapshots of each extension block, which will allow us to look at both the interaction of those within the extension block as well as how the data contained within the extension block is managed. For the sake of simplicity, we will have 5 MWEB users who are pegging-into a freshly created extension block.

User	User Tag	Litecoin Address	Initial Peg-In Amount
Albus	A	LTCA	100
Barty	B	LTCB	50
Cho	C	LTCC	25
Dobby	D	LTCD	7
Emeric	E	LTCE	1000

Let's first begin with the pegging-in process, which will start with each user preparing the bech32 address corresponding to their Litecoin address. These are then collected by a miner, placed on the Hogwarts Express and sent to the Hogwarts Address.

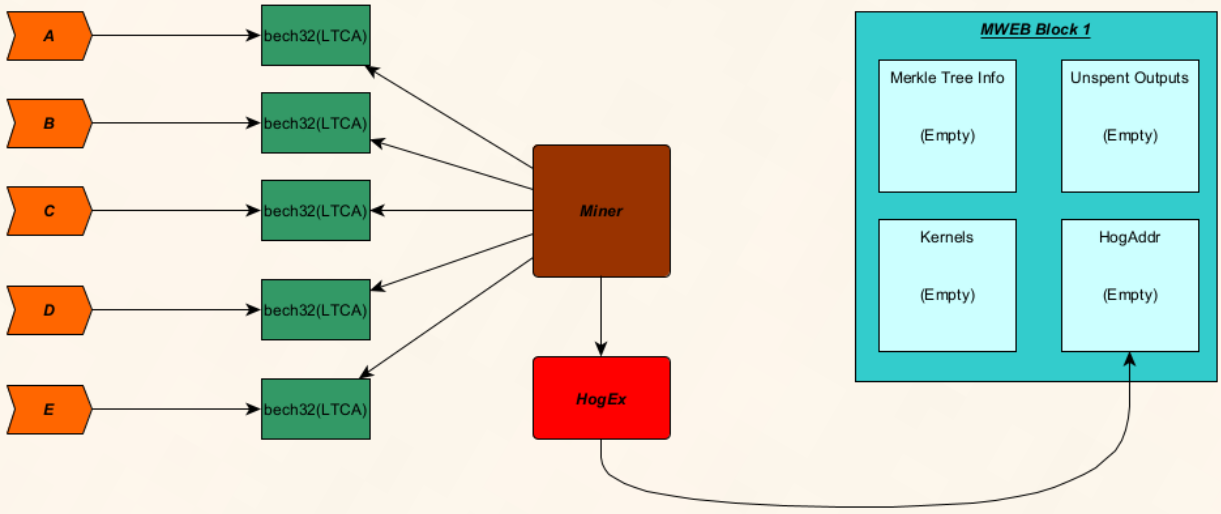


Figure 2: Step 1 - Pegging-In

Once the transaction completes and the block is mined, the second MWEB Block will look like the following:

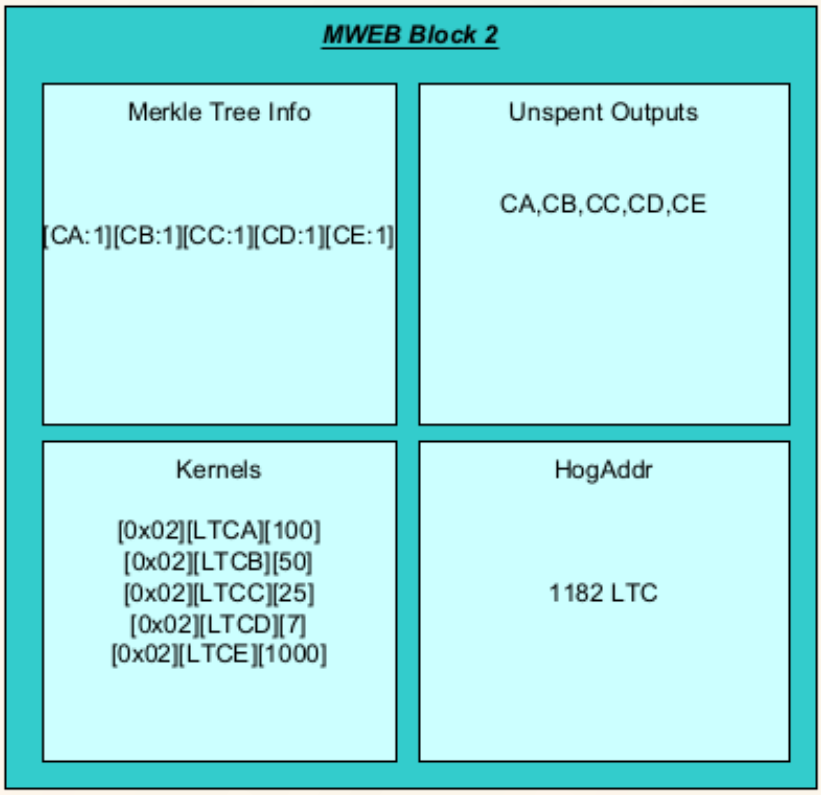


Figure 3: State of MWEB 2

We can see that the unspent outputs now include the 'coins' which are 'minted' to each of the users pegging-in. The Merkle mountain range will include the commitments as the root of their own tree. The amount of Litecoin in the HogAddr is equal to the total amount of the commitments and the kernel contains the transactions (which at this point is only the peg-in transactions). Now, let's consider the following transactions:

1. Albus sends Cho 25 MWEB Litecoin.
2. Emeric sends Barty 300 MWEB Litecoin and pegs-out 500 MWEB Litecoin.
3. Dobby sends Albus 2 MWEB Litecoin.

Behind the scenes, the unspent outputs are used to construct transactions as follows:

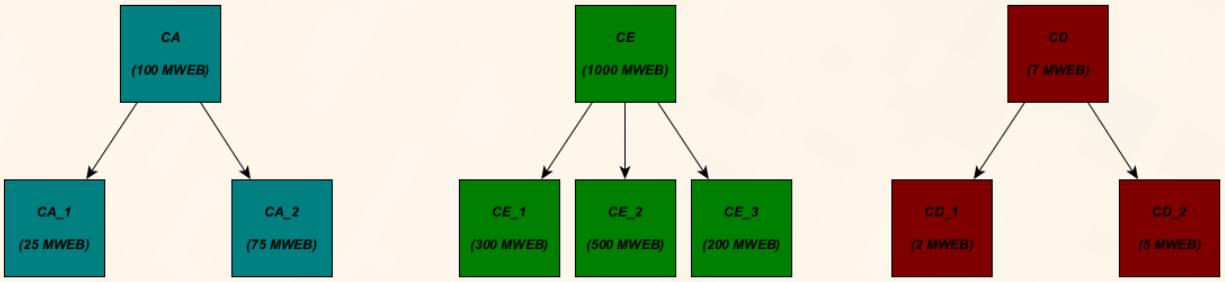


Figure 4: Transaction Graph

Now, the block changes a bit, since the old outputs are now split and the new outputs will take their place, giving us:

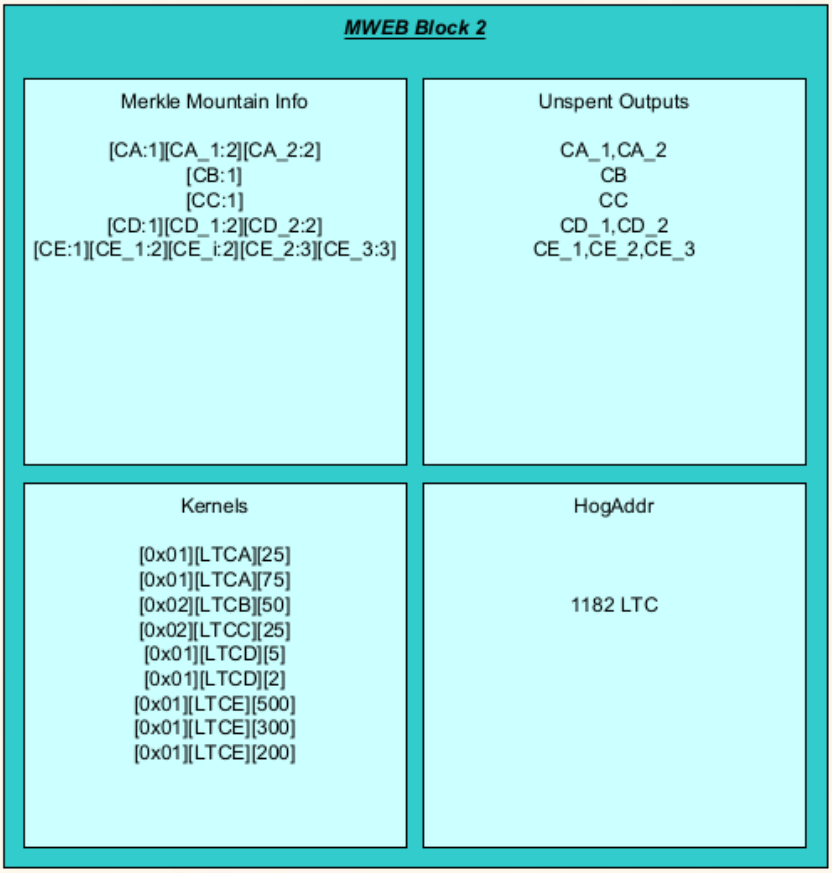


Figure 5: Intermediary State of Block 2

Note that this is just to illustrate what is occurring before the finalization of the block, so this is not quite the end state. The first thing to note is the change of kernels, where the 0x01 tag is being used here for inner block transactions. The transactions are simply from the user to themselves, in order to split the coin. In practice, though, a transaction between two people will have two outputs when the commitment does not

have the exact value the transaction demands. The first output goes to the receiver, and the second output is the change of the transaction.[2]

The second thing to notice is that the unspent outputs have changed as well, with the new coins taking the place of the old ones, and the old ones being removed from the set entirely. This guarantees that Albus can no longer spend CA, since he is now in possession of two coins whose value sums to CA. Lastly, the Merkle mountain range has been appended for the new outputs. We can see that the last entry has a CE_i, which is just a placeholder for the intermediary transaction.

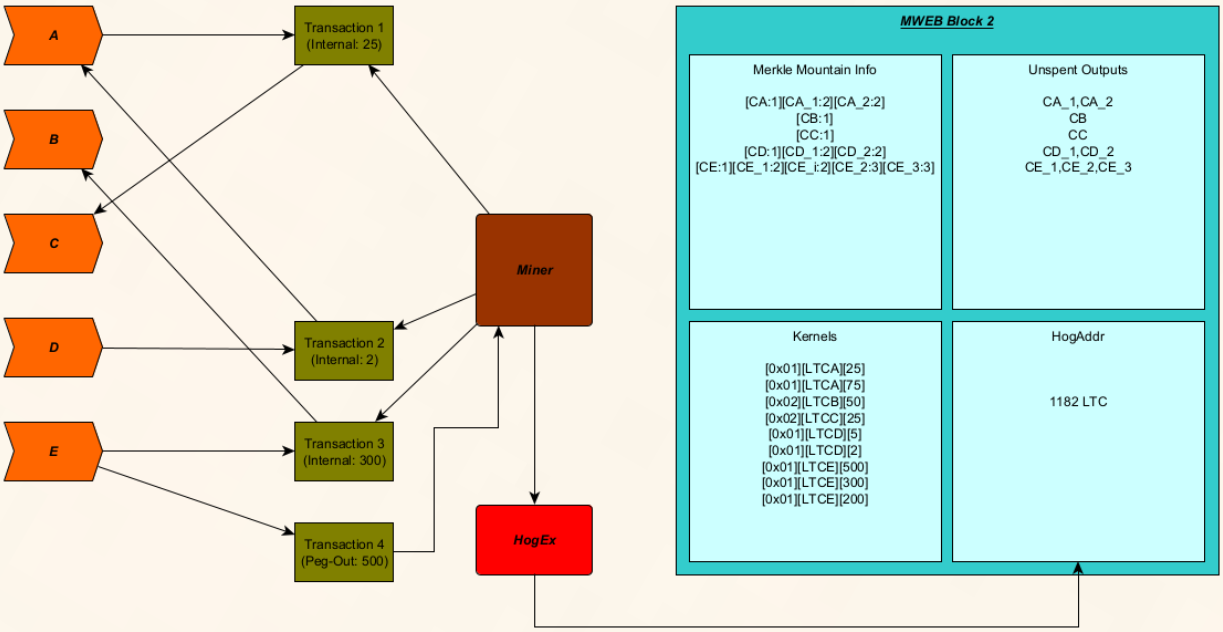


Figure 6: Transactions for Block 2

Once everything is consolidated and mined, we have the final state of Block 2. We can see that the unspent outputs have changed very little: only CE_2 has been removed, as it was the amount that was pegged-out by Emeric. The peg-out request is no longer present in the kernel, but is reflected in the amount remaining in the HogAddr. The kernels have likewise changed to reflect the *change in ownership* of the unspent outputs. One important thing to note here is that a passive observer will see that the amount pegged-out can only be pegged-out by Emeric, since no one else has nearly that much in.

[2]Denoting it in this way doesn't change the protocol that is being used, but it makes the transactions a little easier to understand.

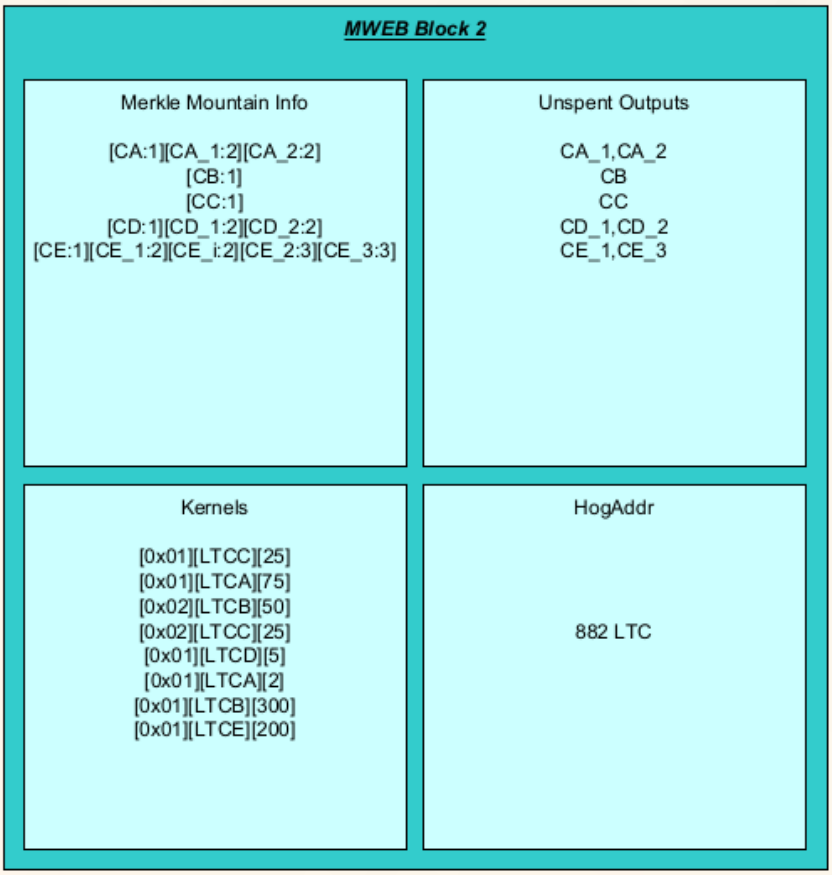


Figure 7: Final State

It is also important to note that from the transaction kernels, we can determine who transacted with who, but we do not know the amounts necessarily. The above diagram is not making use of the stealth addresses, which are implemented in MWEB.

5 Security and Privacy Guarantees

5.1 Inflation Resistance

In a a fiat currency setting, inflation and deflation are inevitable. Government involvement and world events play a major role in altering value of a currency. In the realm of e-coins, inflation resistance means something different. Of course the value of cryptocurrecies can fluctuate, but users mostly want to be assured that money cannot be generated from nothing. This includes things like double-spending attacks. For Bitcoin, or other pseudonymous e-coins, the ledger being fully visible provides that assurance, but the same cannot be said for a privacy-based e-coin.

Recall that the design of MWEB is to add non-interactivity by adding more validation rules. The underlying protocol is still Mimblewimble, which means that the mechanisms which provided inflation resistance for Mimblewimble still apply. These tools are as follows:

- The Mimblewimble balance equation
- Valid signatures for historical kernels
- Valid rangeproofs for all unspent transaction outputs

With the items above, MWEB users can be assured that malicious attackers are prevented from generating invalid MWEB coins.

5.2 Theft Resistance

In an MWEB transaction, the sender is responsible for choosing the blinding factors for the receiver's output. The blinding factor is encrypted into the output and sent via the Elliptic Curve Diffie-Hellman key exchange. This differs from typical Mimblewimble, where everyone chose their own blinding factors.
To recover the blinding factor, the following are required:

- The receiver's subaddress (A_i, B_i)
- Value of the coin, v
- The nonce, n

Under existing Mimblewimble structure, not knowing n already prevents attackers from stealing an output. However, people with access to the receiver's private scan key a, are able to decrypt n from the output. This means that the following people have access to the nonce and blinding factor:

- The receiver of the transaction
- The sender of the transaction (nonce generator)
- Auditors with access to a
- Arbiters that need to verify payment

Obviously, on the first party in the above list should be able to spend the coin, so there needs to be another term that only the receiver should have access to. This takes the form of the subaddress spend key, b_i. We hearken back to Definition 4.7 and rework the equation as follows:

$$\sum K_s + \sum (K_i - K_o) = \sum E' = x' \cdot G$$

The signature on each input proves knowledge of both K_i and K_o and the signature of the output proves knowledge of K_s. Proving knowledge of all private keys prevent rogue key attacks. Indeed, to balance the stealth offset equation, knowledge of the sum of all secret keys for the $K'_o s$ is necessary.

5.3 Horizon Attack

Related to theft resistance, a horizon attack can be attempted if a user knows that coins they sent were not spent after some amount of time. The spender can force a large reorganization beyond the horizon to send the receivers output back to themselves. This works because the sender still knows the blinding factor, but stealth balance equations and input signatures aren't validated for transactions in blocks beyond the horizon.

Such an attack is admittedly, unlikely, but it isn't impossible. Recent headlines surrounding the attack on Monero from the smaller e-coin, Qubic are a direct example of this type of threat being realistic [Rod]. To this vulnerability, we propose the points below:

- The malicious sender has to be assured that the coins they sent have not been spent. This is not trivial enough to take for granted
- The cost to reorganize is **not** insignificant. In addition to normal miners, Litecoin is being ASIC-friendly, which Monero avoids via its Proof-of-Work algorithm RandomX [Tev23]. The place of ASICs in privacy-based cryptocurrency is still being debated, but they do a provide another buffer against a major reorganization. Even to obtain 28% of the hashrate of a major coin (which is closer to what Qubit was actually able to achieve [Wyb25]) would require vast resources and deep coffers.
- If a user is really wary of this particular attack, simply performing a self-spend after receiving a large amount would completely negate this attack, and the cost is only an additional kernel.

5.4 Kernel Malleability

Consider the following scenario: Albus sends an output to Barty, who then then sends the exact output to Cho. If Albus and Cho were to corroborate, they could back-calculate the kernel commitment's blinding factor, and then replace the kernel. Obviously, a one-to-one spending pattern like this is uncommon, and it's more likely that Barty breaks his coins into smaller pieces in a manner outlined in Section 3.4, which means that kernel's blinding factor is only known to him. If the scenario were to be entertained, however, the stealth excess takes care of this. Including it inside of the kernel, and also using it to sign the kernel means that malleating the kernel would invalidate the whole transaction.

A Appendix

Diagram A.1: Interactive Mimblewimble Protocol

Reading from top to bottom should be read as the sequence of events with the middle arrow(s) indicating communication between parties.

Public: $p \in \mathbb{N}$, $G, H \in \mathbb{G}$, $C_i = v_i \cdot H + r_i \cdot G$
Private: $v_i, r_i \in \mathbb{F}_p$, where $v_i \in [0,\ 2^n - 1]$

Prover		**Verifier**
Determine $v_o < v_i$ to pay out		
Set $v_c = v_i \ - v_o$		
Sample $r_c \xleftarrow{\$} \mathbb{F}_p$		
Construct $C_c = v_c \cdot H + r_c \cdot G$		
Select nonce n_c and offset o_c		
Define $N_c = n_c \cdot G$	$\xrightarrow{C_i, C_c, v_o, o_c N_c}$	
		Sample $r_o \xleftarrow{\$} \mathbb{F}_p$
		Construct $C_o = v_o \cdot H + r_o \cdot G$
		Construct range proof π_o
		Select nonce n_d and offset o_d
		Set $o = o_c + o_d$
		Define kernel public key, $K = C_o + C_c - C_i - o \cdot G$
		Define $N_d = n_d G$
		Set $N = N_c + N_d$
		Construct partial signature,
	$\xleftarrow{C_o, \pi_o, N_d, o_d, \sigma_d}$	$\sigma_d = n_d + (r_o - o_d)\mathcal{H}(N, K)$
Set $o = o_c + o_d$		
Complete signature, $\sigma = \sigma_d + n_c + (r_c - r_i - o_c)\mathcal{H}(N, K)$		
Set $\mathsf{tx} = (C_i, C_o, C_c, \pi_d, K, (N, \sigma), o)$	$\xrightarrow{\mathsf{tx}}$	
		Check $C_o + C_c - C_i \stackrel{?}{=} K + o \cdot G$
		Check $\sigma \cdot G \stackrel{?}{=} N + K \cdot \mathcal{H}(N, K)$

References

[ANWOW13] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O'Hearn, and Christian Winnerlein. BLAKE2: simpler, smaller, fast as MD5. Cryptology ePrint Archive, Paper 2013/322, 2013. https://eprint.iacr.org/2013/322.

[BBB+17] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Paper 2017/1066, 2017. https://eprint.iacr.org/2017/1066.

[BDH+] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Keccak. https://keccak.team/keccak.html. Accessed: 2025-11-17.

[Bur20] David Burkett. Offline transactions in mimblewimble. https://gist.github.com/DavidBurkett/32e33835b03f9101666690b7d6185203, 2020. Accessed: 2025-11-8.

[Bur21] David Burkett. One-sided transactions in mimblewimble (consensus layer). https://github.com/DavidBurkett/lips/blob/master/lip-0004.mediawiki, 2021. Accessed: 2025-11-8.

[Bur25] David Burkett. Personal communication, 2025.

[CHJ+20] Heewon Chung, Kyoohyung Han, Chanyang Ju, Myungsun Kim, and Jae Hong Seo. Bulletproofs+: Shorter proofs for privacy-enhanced distributed ledger. Cryptology ePrint Archive, Paper 2020/735, 2020. https://eprint.iacr.org/2020/735.

[EKRN22] Liam Eagen, Sanket Kanjalkar, Tim Ruffing, and Jonas Nick. Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments. Cryptology ePrint Archive, Paper 2022/510, 2022. https://eprint.iacr.org/2022/510.

[ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In George Robert Blakley and David Chaum, editors, *Advances in Cryptology*, pages 10–18, Berlin, Heidelberg, 1985. Springer Berlin Heidelberg.

[Epi] EPIC CASH. https://epiccash.com/. Accessed: 2025-11-28.

[FO22] Georg Fuchsauer and Michele Orrù. Non-interactive mimblewimble transactions, revisisted. Cryptology ePrint Archive, Paper 2022/265, 2022. https://eprint.iacr.org/2022/265.

[Gri] GRiN. https://grin.mw/. Accessed: 2025-11-28.

[Jed16] Tom Elvis Jedusor. Mimblewimble. Anonymous, 2016. docs.beam.mw/Mimblewimble.pdf.

[jo20] joshtriplett and oconnor663. the official rust and c implementations of the blake3 cyrptographic hash function. https://github.com/BLAKE3-team/BLAKE3, 2020. Accessed: 2025-12-3.

[MGGR] Ian Miers, Christina Garman, Matthew Green, and Aviel Rubin. Zerocoin: Anonymous Distributed E-Cash from Bitcoin. https://www.allcryptowhitepapers.com/zcoin-whitepaper/. Accessed: 2024-11-13.

[Mil] Jim Miller. The Frozen Heart Vulnerability in Bulletproofs. https://blog.trailofbits.com/2022/04/15/the-frozen-heart-vulnerability-in-bulletproofs/. Accessed: 2025-12-5.

[MMR] Merkle Mountain Ranges (MMR). https://docs.grin.mw/wiki/chain-state/merkle-mountain-range/. Accessed: 2025-12-6.

[Noe15] Shen Noether. Ring signature confidential transactions for monero. Cryptology ePrint Archive, Paper 2015/1098, 2015. https://eprint.iacr.org/2015/1098.

[Poe16] Andrew Poelstra. Mimblewimble, 2016. https://download.wpsoftware.net/bitcoin/wizardry/mimblewimble.pdf.

[Pol10] By J. M. Pollard. Monte carlo methods for index computation (mod p). In *American Mathematical Society: Mathematics of Computation*, 2010. https://api.semanticscholar.org/CorpusID:235457090.

[Rod] Francisco Rodrigues. Monero Suffers Deepest-Ever Blockchain Reorganization, Inalidating 118 Transactions. https://www.coindesk.com/web3/2025/09/15/monero-suffers-deepest-ever-blockchain-reorganization-invalidating-118-transactions. Accessed: 2025-12-8.

[Tev21] Tevador. Minglejingle: Scalable blockchain with non-interactive transactions. https://gist.github.com/tevador/f3a66a2f15a8a3a04a1dde1ea65f9205, 2021. Accessed: 2025-11-8.

[Tev23] Tevador. RandomX. https://github.com/tevador/RandomX, 2023. Accessed: 2025-12-8.

[Wyb25] Shai Wyborski. The Qubic Minority Report. https://shai-deshe.gitbook.io/parallel-thoughts/proof-of-work/the-qubic-minority-report, 2025. Accessed: 2025-12-8.

[Yap25] Reuben Yap. Binance halts firo trading - withdraw your firo now. https://firo.org/nl/2025/04/16/binance-delisting.html, 2025. Accessed: 2025-12-3.

[Yu20] Gary Yu. Mimblewimble non-interactive transaction scheme. Cryptology ePrint Archive, Paper 2022/265, 2020. https://https://eprint.iacr.org/2020/1064.